

IF231303 Arsitektur Perangkat Lunak

Pertemuan 03: Model-View-*

September 6, 2026

Alfa Yohannis



Tujuan Pembelajaran

- 1 Menjelaskan pembagian peran pada MVC, MVP, MVVM, dan MVI beserta arah ketergantungannya.
- 2 Membedakan keempat varian berdasarkan komponen yang dikenal View.
- 3 Mengukur ketergantungan, kemudahan pengujian, dan luas dampak perubahan.

- Antarmuka pengguna berubah jauh lebih sering daripada aturan bisnis.
- Warna, susunan tombol, dan urutan layar dapat berganti beberapa kali dalam satu semester.
- Aturan perhitungannya tetap, sehingga keduanya sebaiknya dipisahkan.

Seluruh varian Model-View-* memisahkan keduanya dengan cara yang sama. Yang membedakan hanyalah cara bagian penghubung bekerja, dan komponen mana yang boleh dikenal oleh View.

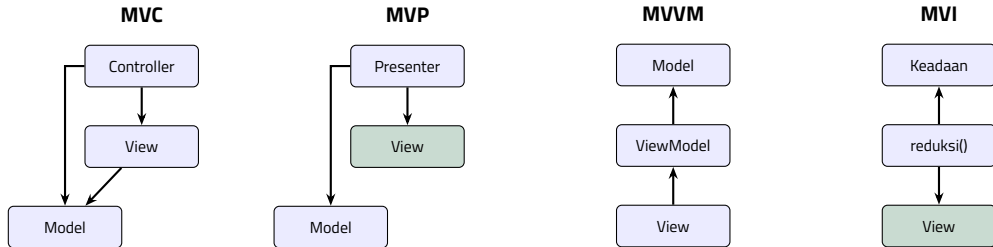
Bab ini membandingkan MVC, MVP, MVVM, dan MVI memakai satu aplikasi yang sama, sehingga perbedaannya dapat diukur.

Tiga Peran yang Sama

Peran	Tanggung jawab
Model	Menyimpan data dan menjalankan aturan bisnis
View	Menampilkan data dan menerima tindakan pengguna
Penghubung	Menerjemahkan tindakan pengguna menjadi perubahan pada Model

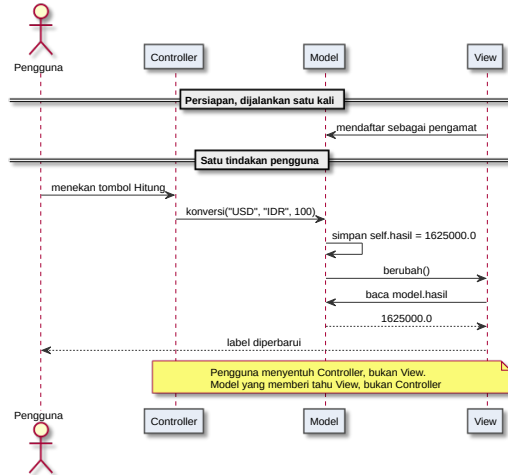
Nama penghubungnya berbeda, yaitu Controller, Presenter, ViewModel, atau sebuah fungsi reduksi. Yang menentukan adalah arah ketergantungannya.

Arah Ketergantungan Keempat Varian

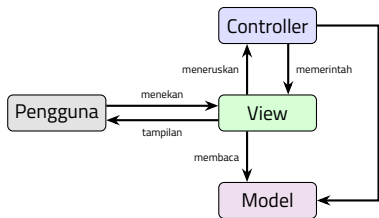


Kotak hijau menandai View yang tidak bergantung pada komponen mana pun.

MVC Klasik: Model yang Memberi Tahu



MVC Masa Kini: View Membaca Model

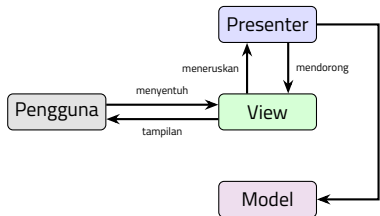


- Pengguna menyentuh View, lalu View meneruskannya ke Controller.
- Controller memanggil Model, lalu memerintah View menggambar.
- View membaca Model sendiri untuk mengambil nilainya.
- Ciri khasnya, **View mengenal Model**.

```
class View:
    def __init__(self, induk, model):
        self.model = model    # ciri khas MVC
    def saat_ditekan(self):
        self.controller.tangani_masukan(...)
```

Total ketergantungan 3, dan View butuh Model.

MVP: View Dibuat Pasif

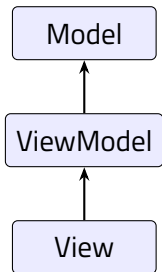


- Pengguna menyentuh View, lalu View meneruskannya ke Presenter.
- Presenter menyusun seluruh teks tampilannya.
- View hanya menyimpan apa yang diberikan Presenter.
- **View tidak mengenal Model** sama sekali.

```
class View:
    def tampilkan(self, teks):
        self.teks = teks
```

Total ketergantungan 2, View butuh nol. Pengujian cukup memakai View tiruan.

MVVM: View Mengamati Properti

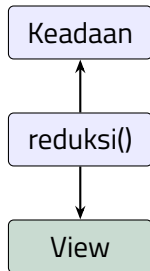


- ViewModel menulis hasil ke properti yang dapat diamati.
- View mendaftarkan sebagai pengamat, lalu memperbarui dirinya.
- **ViewModel tidak mengenal View**, sehingga arahnya satu.

```
class View:  
    def __init__(self, view_model):  
        view_model.teks.amati(self.saat_berubah)
```

Total ketergantungan 3, sama dengan MVC, tetapi arahnya berbeda. Pengujian cukup membaca propertinya.

MVI: Penghubungnya Sebuah Fungsi



- Setiap tindakan pengguna dibungkus menjadi Intent.
- Fungsi reduksi mengubah keadaan lama menjadi keadaan baru.
- Keadaannya tidak dapat diubah, sehingga riwayat dapat diputar ulang.

```
def reduksi(keadaan, intent):  
    hasil = domain.hitung_konversi(...)  
    return replace(keadaan, teks=...)
```

Total ketergantungan 0, sebab penghubungnya fungsi, bukan kelas yang menyimpan rujukan.

Siapa Memakai Varian yang Mana

Varian	Contoh produk
MVC klasik	Smalltalk-80
MVC masa kini	Ruby on Rails, Laravel, Spring MVC, ASP.NET Core MVC
MVP	Google Web Toolkit, Windows Forms, Android sebelum Jetpack
MVVM	Windows Presentation Foundation, .NET MAUI, Vue.js versi awal
MVI	Redux, arsitektur Elm, Flutter Bloc

Tiga Sebutan yang Sering Keliru

- **Django bukan MVC menurut namanya sendiri.** Django menyebut polanya MTV, yaitu *Model-Template-View*. Berkas `views.py` berperan sebagai Controller, sedangkan templatnya berperan sebagai View.
- **Android tidak menyebut dirinya MVVM.** Dokumentasi resminya menyebut `ViewModel` sebagai *state holder* di dalam susunan berlapis, dan istilah MVVM tidak dipakai sama sekali.
- **Redux mewarisi arsitektur Elm.** Fungsi pembaruan pada Elm bertanda tangan $(action, state) \Rightarrow state$, dan fungsi tersebut sama perannya dengan fungsi reduksi.

Penyebutan pola berubah seiring versi. Rujukan versi karena itu perlu ikut dicatat.

Satu Gagasan Bernama Intent

Intent untuk mengirim pesan singkat tidak mengirim apa pun sendiri. Objek tersebut hanya menyatakan niat.

Hal	Intent pada Android	Intent pada MVI
Gagasannya	Niat dibungkus menjadi objek data	Niat dibungkus menjadi objek data
Tujuannya	Ada keadaan berubah atau proses dijalankan	Ada keadaan berubah atau proses dijalankan
Penyalurnya	Sistem operasi Android	Fungsi reduksi
Cakupannya	Melintasi komponen dan aplikasi	Di dalam satu layar

Gagasannya sama, dan namanya *command*. Yang berbeda hanya tingkat penerapannya, sehingga memakai *Intent* belum berarti memakai MVI.

Analogi: Dua Petugas Loket

- Petugas pertama harus membuka lemari arsip sendiri untuk menjawab.
- Petugas kedua hanya membacakan lembar yang sudah disiapkan bagian lain.
- Petugas pertama sulit digantikan, sebab penggantinya harus mengenal isi lemari.
- Petugas kedua mudah digantikan siapa pun.

View pada MVC berperan seperti petugas pertama, sedangkan View pada MVP dan MVI berperan seperti petugas kedua.

Kasus: Menguji Layar yang Tidak Dapat Dijalankan

- 1 Tim ingin menguji aturan penolakan nominal nol.
- 2 Pada MVC, pengujian tetap menuntut objek View dibuat lebih dahulu.
- 3 View menuntut Model, dan Model menuntut sambungan data.
- 4 Pengujian akhirnya menyalakan hampir seluruh aplikasi.
- 5 Pada MVI, pengujian cukup memanggil satu fungsi reduksi.

Yang menentukan bukan jumlah kelasnya, melainkan komponen apa yang dituntut View.

Hasil Pengukuran

Varian	Total	View butuh	Cara pengujian
MVC	3	Model	Membutuhkan View asli
MVP	2	tidak ada	Cukup View tiruan
MVVM	3	ViewModel	Tanpa View, cukup membaca propertinya
MVI	0	tidak ada	Tanpa View, cukup memanggil fungsi reduksi

MVC dan MVVM mencatat total yang sama, tetapi arahnya berbeda. Angka perlu dibaca bersama arahnya.

Ketika Satu Komponen Menggemuk

Pemisahan peran hanya bertahan bila setiap komponen menolak pekerjaan yang bukan miliknya.

```
class Controller:                # fat controller
    def tangani_masukan(self, asal, tujuan, nominal):
        if nominal <= 0:          # aturan bisnis
            return "Galat: nominal harus positif"
        hasil = nominal * self.model.cari_kurs(asal, tujuan)
        return f"Hasil: {hasil:,.2f}" # penyusunan tampilan
```

- Aturan tersebut tidak dapat dipakai ulang oleh pintu masuk lain.
- Pengujiannya menuntut konteks permintaan, padahal isinya hanya perkalian dan perbandingan angka.

Tiga Gejala dan Perbaikannya

Gejala	Tanda yang terlihat	Perbaikan
Fat controller	Validasi dan perhitungan ada di Controller	Pindahkan aturannya ke Model
Fat model	Satu kelas mengurus aturan, basis data, dan format	Bagi menurut alasan berubahnya
Fat view	Percabangan dan perhitungan ada di templat	Susun teksnya di Presenter

`periksa_mv.py` mengukur arah ketergantungan, bukan ukuran komponennya. Arah panah yang benar belum menjamin pembagian kerja yang sehat.

- Seluruh varian memisahkan tampilan dari aturan bisnis. Yang berbeda hanyalah cara penghubungnya bekerja.
- Pembeda yang menentukan adalah komponen yang dikenal View. Semakin sedikit yang dikenal, semakin mudah diuji tanpa antarmuka.
- Jumlah ketergantungan saja tidak cukup, sebab arahnya ikut menentukan.
- Arah panah yang benar pun belum menjamin pembagian kerja yang sehat, sebab satu komponen tetap dapat menggemuk.

Persiapan Latihan

```
# sekali saja, dari akar repositori
bash sources/siapkan_venv.sh
source .venv/bin/activate
python -c "import tkinter"

cd sources/chapter03
python mvc.py
```

Latihan bab ini menampilkan jendela, sehingga modul `tkinter` harus tersedia. Pada Debian dan Ubuntu, modulnya ada di paket `python3-tk`. Satu aplikasi, empat varian, tiga masalah berbeda.

Latihan 1: Memetakan Ketergantungan

```
python periksa_mv.py
```

- 1 Catat baris Total ketergantungan dan Ketergantungan View untuk keempat varian.
- 2 Buka keempat berkas, lalu tunjukkan baris konstruktor yang menjadi sumber setiap ketergantungan.
- 3 Gambarkan arah panah antar komponen untuk keempat varian.

Pertanyaan: MVC dan MVVM mencatat total yang sama. Mengapa keduanya tetap berbeda sifatnya?

Latihan 2: Menguji Tanpa Antarmuka

```
python uji_tanpa_antarmuka.py
```

- 1 Catat cara uji yang dipakai setiap varian.
- 2 Bandingkan isi keempat fungsi ujinya.
- 3 Coba tulis ulang pengujian MVC tanpa membuat objek View, lalu catat apa yang menghalangi.

Pertanyaan: cocokkan kolom cara uji dengan kolom ketergantungan View pada Latihan 1. Apa hubungan keduanya?

Latihan 3: Mengukur Dampak Perubahan

```
git diff --stat mvc.py mvp.py mvvm.py mvi.py
```

- 1 Catat perkiraan berkas yang akan berubah sebelum mulai menyunting.
- 2 Tambahkan baris kedua yang menyebut nilai kurs pada keempat varian.
- 3 Jalankan ulang pengujian untuk memastikan keempatnya masih berjalan.
- 4 Hitung berkas, kelas, dan baris yang berubah.

Pertanyaan: mengapa varian dengan ketergantungan paling sedikit belum tentu yang paling sedikit berubah?